Arqon ContextOS Natural Flow User Stories

Narrative context artifact for rebuilding PM/Coder/Helper/Auditor workflow context

Status: REQUIRES_HUMAN_REVIEW | development diagnostic only | NOT SEALED-TEST CERTIFIED | not promotable

Status:

- REQUIRES_HUMAN_REVIEW
- development diagnostic only
- NOT SEALED-TEST CERTIFIED
- not promotable

Purpose:

This document preserves the user stories, design intent, workflow expectations, command semantics, and natural-flow doctrine shared during the Arqon ContextOS design session. It is intended to help rebuild context in a new PM/Coder/Auditor/Helper AI session without relying on chat history.

Boundary:

This document is a narrative context artifact. It is not a certification artifact, not a promotion manifest, not a sealed-test certification, and not a production approval.

1. Core Problem Being Solved

The central problem is not merely that GPTs need access to files. The deeper problem is that every new GPT session starts without the working state of the project, forcing the human operator to manually reconstruct context every day. The user has had to spend time each morning choosing which pieces of context to assemble, what order to present them in, which governance facts matter, which recent decisions are still active, and what each role needs to know. This manual context rebuilding is slow, fragile, and prone to omission.

The desired solution is a repo-backed shared state system where each role GPT can load the current project state through a short command. The GPTs should not rely on chat memory, copy/paste, or fragile human recollection. The repo should become durable memory. Cloudflare should act as the broker. GitHub should hold the canonical artifacts. Each role GPT should have a constrained key and should be able to load only its own role context unless explicitly authorized.

The goal is not full autonomous control. The human still opens the GPTs, issues commands, monitors completion, and decides how to proceed. The key improvement is that the human should no longer have to copy and paste every PM spec, Coder response, Helper result, or Auditor report from one chat to another. The system should persist these handoffs through the repo and Cloudflare broker.

User story

As the human operator, I want each GPT to load the current Arqon Zero state from the repo through a simple command, so I do not have to rebuild context manually every day.

Acceptance criteria

- PM GPT can load PM-specific context.
- Coder GPT can load Coder-specific context.

- Auditor GPT can load Auditor-specific context.
- Role isolation works.
- Repo state, not chat memory, is treated as the durable continuity layer.
- Required conservative status labels remain present.

2. Human-Orchestrated, Not Autonomous-by-Default

The user does not want GPTs autonomously looping in the background without oversight. The preferred model is a human-orchestrated system where the human opens the PM GPT, Coder GPT, Auditor GPT, or Codex Helper as needed and issues concise commands. The GPTs use Cloudflare and GitHub to save and load state, but the human remains the conductor.

This is different from a fully autonomous agent swarm. The human decides when to start a flow, when to move to the next role, when to ask for an audit, when to continue iteration, when to stop, and when to approve advancement. The system should remove context-passing friction, not remove human judgment.

User story

As the human operator, I want to manually open and command each GPT, while the GPTs persist their state and artifacts through ContextOS, so I can keep control without being buried in copy/paste work.

Acceptance criteria

- GPTs do not require always-on push connections.
- Cloudflare stores and serves current state.
- GitHub stores persistent artifacts.
- GPTs pull current state when commanded.
- Human final authority is preserved.
- No AI self-certifies or auto-promotes.

3. The Repo as Durable Memory

The project repo should become the memory and evidence ledger. ArqonZero contains project-specific context, notes, messages, flows, artifacts, and decisions. ArqonContextOS contains the reusable infrastructure: Worker broker, OpenAPI schema, command definitions, project templates, and docs.

The Cloudflare Worker acts as the ContextOS broker. It reads and writes allowlisted paths in the project repo through a GitHub App. Each GPT gets a role-specific bearer key. The broker uses that key to infer the role and enforce permissions.

User story

As the system designer, I want ContextOS to store notes, messages, flows, artifacts, and decisions in GitHub, so the project has durable, auditable memory independent of any single GPT chat.

Acceptance criteria

- ArqonContextOS remains infrastructure.
- ArqonZero remains the project memory/artifact repo.
- GPTs access ArqonZero through Cloudflare broker, not arbitrary GitHub browsing.
- Broker writes only to allowlisted governance paths.

- Secrets are never stored in repo files or GPT Knowledge.

4. Context Notes and Messages Are Informal

The current v0.2 ContextOS layer supports context notes and role messages. These are important but informal. A context note captures useful thinking, suggestions, analysis, design ideas, or observations. A message sends one role's note or request to another role's inbox.

A note or message must not be treated as an official PM spec, Coder patch bundle, Helper execution report, Auditor verdict, or human decision. PM can later incorporate a useful note into a formal spec, but the note itself is not the official artifact.

User story

As the human operator, I want Coder or Auditor suggestions to be saved as context notes and sent to PM's inbox, so useful ideas are not lost and do not need to be manually pasted between GPTs.

Acceptance criteria

- `/save-context` saves non-official context.
- `/send-message` sends a non-official role-to-role message.
- `/inbox` lets each GPT view its own inbox.
- `/open-message` opens a message.
- `official_artifact: false` is preserved for notes/messages.
- PM must explicitly promote useful ideas into formal PM artifacts.

5. Why "Runs" Were Replaced by "Flows"

The original command idea was `/create-run`, `/load-run`, and `/write-artifact`. That model was too narrow. A "run" sounds like a single task or execution attempt. The user's real workflow is broader: it includes phases, milestones, audits, Coder/Helper iteration loops, audit points, and human advancement decisions.

The better abstraction is a "flow." A flow is a container for a natural chain of work. It can represent a phase, milestone, audit, repair loop, or sprint. A flow can contain multiple phases, artifacts, messages, Helper reports, Auditor findings, and human decisions.

User story

As the human operator, I want to create named flows instead of isolated runs, so a planned sprint, milestone, or audit sequence can have one durable home in the repo.

Acceptance criteria

- User-facing command is `/create-flow`, not `/create-run`.
- User-facing command is `/load-flow`, not `/load-run`.
- Flow folders hold PM plan, milestones, definitions of done, audit points, Coder artifacts, Helper evidence, Auditor reports, and human decisions.
- The system still may use IDs internally, but the user can load flows by a short human-friendly name.

6. Flow Names Are Human-Friendly Aliases

The `name` parameter in `/create-flow` is not just a label. It is a short, easy-to-remember alias that the human can use instead of a long, hard-to-remember flow ID.

Example:

```
/create-flow type=audit name="flow00"
```

The system may create an internal ID such as `FLOW-2026-0001`, but the user should be able to load the flow using:

```
/load-flow name="flow00"
```

or, when useful:

```
/load-flow flow_id="FLOW-2026-0001"
```

User story

As the human operator, I want to give each flow a short name like `flow00`, so I can load and reference active work without copying long IDs.

Acceptance criteria

- `name` is unique among active flows.
- `name` maps to a canonical internal `flow_id`.
- User can load by `name` or `flow_id`.
- Duplicate active names fail closed.
- Broker maintains a `flow_index.json` or equivalent mapping.

7. Command Defaults Should Reduce Typing

The user does not want to type `project=ArqonZero` in every command because these GPTs are built specifically for Arqon Zero. The project should default to ArqonZero.

The user also does not want to type `role=CODER_AI` while using the Coder GPT. The role should be inferred from the broker key. PM GPT implies `PM_AI`; Coder GPT implies `CODER_AI`; Auditor GPT implies `AUDITOR_AI`.

Parameters such as `project`, `role`, and `type` can still exist internally or for explicit override cases, but they should not be required in normal use.

User story

As the human operator, I want commands to infer project and role from the GPT I am using, so I can type short commands like `/load-flow name="flow00"` instead of long boilerplate commands.

Acceptance criteria

- `project` defaults to ArqonZero.
- `role` is inferred from authenticated broker key.
- `type` defaults safely where appropriate, likely to `audit` when creating a flow.
- User can specify role only when intentionally requesting another role's view, and the broker enforces access.
- Conflicting role requests fail with `ROLE_MISMATCH`.

8. Final Core Command Vocabulary

The current and planned command system should be concise and human-friendly.

Implemented v0.2 commands:

- `/sync-context`
- `/sync-constitution`
- `/save-context`
- `/send-message`
- `/inbox`
- `/open-message`
- `/archive-message`

Next flow commands:

- `/create-flow`
- `/load-flow`
- `/flow-status`
- `/adv-flow`

Backend/manual primitive:

- `/write-artifact`

The user should rarely type `/write-artifact` directly. Instead, PM, Coder, Helper, and Auditor GPTs should write their official artifacts automatically as part of `/create-flow`, `/load-flow`, or their role-specific completion behavior.

User story

As the human operator, I want the command set to reflect how I actually work, so I can create flows, load flows, check status, and advance flows without typing repetitive artifact-write commands.

Acceptance criteria

- `/create-flow` creates the flow and writes PM artifacts automatically.
- `/load-flow` loads flow state for the current GPT role.
- `/flow-status` summarizes progress.
- `/adv-flow` advances the flow after gate criteria are met.
- `/write-artifact` exists as a lower-level primitive but is not the normal user command.

9. The Three Flow Types

The user identified three natural flow types: phase, milestone, and audit. They differ in how much PM or Auditor involvement is expected.

Phase Flow

A phase flow keeps PM closely involved. It is useful when the design is uncertain, the architecture is changing, or the human wants PM to remain in the loop after each phase.

Natural pattern:

1. PM plans the phase.
2. Coder implements the phase.
3. Helper executes and reports.
4. Coder repairs if needed.
5. PM decides the next phase.

Milestone Flow

A milestone flow is faster. PM creates the full plan and milestones up front. Coder and Helper iterate through a milestone chunk without PM involvement at every small step. Auditor is used only if requested or triggered.

Natural pattern:

1. PM creates phased plan and milestone list.
2. Human decides to implement a milestone chunk.
3. Coder loads the flow.
4. Coder + Helper iterate until the milestone chunk is complete.
5. PM receives completion report.
6. Auditor may be invoked if needed.

Audit Flow

An audit flow is the safest default for governance-sensitive work. PM defines plan, milestones, audit points, gate criteria, and definitions of done. Coder and Helper move quickly within the milestone chunk, while Auditor receives evidence and gates milestone/audit points.

Natural pattern:

1. PM creates phased plan, milestones, definitions, audit points, and gate criteria.
2. Coder implements the milestone chunk.
3. Helper executes and reports to both Coder and Auditor.
4. Coder iterates with Helper.
5. Auditor gates the milestone at audit point.
6. Human approves advancement or sends it back for repair.

User story

As the human operator, I want to choose between phase, milestone, and audit flows, so I can decide whether PM, Coder/Helper, or Auditor should dominate the next stretch of work.

Acceptance criteria

- `type=phase` keeps PM closely involved.
- `type=milestone` lets Coder + Helper move fast through a PM-defined milestone chunk.
- `type=audit` includes audit points and Auditor gating at critical moments.
- `type` is short, not `flow-type`.
- Default type should be safe, likely `audit`.

10. Avoiding the Auditor Bottleneck

The user identified a serious problem: if every workflow is forced through PM → Coder → Helper → Auditor, development becomes too slow because Auditor is slow and deep by design. Auditor should not be in the inner loop for every tiny coding iteration.

Instead, the natural fast loop should usually be:

1. PM creates the plan.
2. Coder implements.

3. Helper executes.

4. Helper reports to Coder immediately.

5. Coder decides whether to iterate.

6. Auditor receives evidence or is called at audit points, not necessarily every iteration.

This preserves speed while still giving Auditor access to evidence when it matters.

User story

As the human operator, I want Coder and Helper to iterate quickly without waiting for Auditor every time, so the project can move at development speed while still preserving auditability.

Acceptance criteria

- Helper reports to Coder by default.
- Helper also writes evidence for Auditor or sends Auditor a message when appropriate.
- Auditor is blocking only at audit points, hard-fail triggers, critical subsystem points, or claim/promotion gates.
- Milestone and audit flows can include Auditor without making Auditor the inner-loop bottleneck.

11. PM GPT's Role in Flow Creation

The PM GPT is responsible for turning discussion about vision, mission, objectives, and the day's sprint into a concrete phased plan. PM should create the whole plan and milestone structure in one shot when appropriate.

The PM plan should include:

- vision
- mission
- objectives
- phased plan
- milestone list
- project specs
- definitions
- definition of done
- audit points
- risk classification
- gate criteria
- allowed files
- forbidden files
- Coder handoff
- Auditor brief

After discussing the plan, the human chooses the flow type and issues:

```
/create-flow type=audit name="flow00"
```

PM GPT should then save the plan and related artifacts automatically through the Cloudflare Worker. The human should not need to issue a separate `/write-artifact` command.

User story

As the human operator, I want PM GPT to produce a complete phased plan with definitions of done, milestones, and audit criteria, then create the flow in one command, so Coder can load it and begin implementation without another handoff step.

Acceptance criteria

- `/create-flow` works only for PM GPT or authorized human/admin context.
- PM artifacts are written automatically during flow creation.
- PM response includes flow name, flow ID, type, current status, next actor, and load command.
- PM plan includes enough detail for Coder and Auditor to act without re-asking the PM for every phase.

12. Coder GPT's Role After `/load-flow`

The Coder GPT should not need the PM to restate the plan. When the human opens Coder GPT and runs:

```
/load-flow name="flow00"
```

Coder should load the flow documents and produce the implementation package. The user expects Coder to one-shot all code, docs, tests, Helper prompts, expected commands, and SHA/hash expectations where practical.

Coder should automatically upload its outputs to the repo through the Cloudflare Worker. The human should not need to issue `/write-artifact` afterward.

Coder should produce:

- implementation plan
- code/docs/tests patch bundle
- Helper/Codex execution packet
- expected command list
- evidence requirements
- SHA/hash manifest where practical
- repair logic for likely Helper failures

User story

As the human operator, I want Coder GPT to load a named flow, understand the PM plan, generate the implementation bundle, and persist its artifacts automatically, so I can move straight to Helper/Codex execution in VS Code.

Acceptance criteria

- Coder can load by `name`.
- Coder role is inferred.
- Coder reads PM plan, milestones, definitions of done, and audit points.
- Coder writes official Coder artifacts automatically.
- Coder produces a Helper packet that is directly executable by Helper/Codex.

13. Helper/Codex in VS Code

The Helper/Codex step is still somewhat fuzzy, but the preferred direction is clear: Helper should read the repo, not rely on chat copy/paste. Coder should write a Helper packet into the flow folder. The human can then open

VS Code/Codex and issue a short instruction to load the flow, read the latest Helper packet, execute it, and write the report.

Helper/Codex should:

1. pull or read current repo state
2. resolve the flow name through `flow_index.json`
3. read `coder_handoff_to_helper.md`
4. apply patch or perform scoped edits
5. run required commands
6. capture logs, exit codes, hashes, and evidence
7. write `helper_execution_report.md`
8. write `evidence_manifest.json`
9. report to Coder for iteration
10. notify Auditor or write evidence stream for audit points

The Helper packet must explicitly define commit/push authority. Default should be no commit and no push unless the flow artifact explicitly authorizes it.

User story

As the human operator, I want Helper/Codex in VS Code to load the flow's Helper packet from the repo and execute it exactly, so the Coder/Helper loop can proceed without manual copy/paste and without giving Helper design authority.

Acceptance criteria

- Helper reads the latest Coder handoff from the flow folder.
- Helper performs only scoped execution and micro-edits.
- Helper writes execution report and evidence manifest.
- Helper reports to Coder immediately.
- Helper evidence is available to Auditor.
- Commit/push authority is explicit.
- Unauthorized changes stop the flow.

14. Auditor's Role in Natural Flows

Auditor should be used when appropriate, not as an unavoidable bottleneck after every Helper run. Auditor is ideal for generating or validating guardrails, tripwire harnesses, policies, ops guards, evidence checks, and deception-resistance controls.

Auditor should be active in audit flows, at milestones, and whenever the system reaches a critical point. Auditor should check whether code, reports, and results can be adversarially attacked, whether evidence is deceptive, whether claims exceed evidence, and whether tripwires and policies are present.

Auditor should not merely assign a numeric score. The user does not want an arbitrary "85/100" threshold. Instead, Auditor should apply logical gate levels and explicit checklists.

User story

As the human operator, I want Auditor to gate milestones, claims, and risky subsystems using explicit evidence requirements and guardrails, so the system can detect deception and adversarial weaknesses without slowing every small coding iteration.

Acceptance criteria

- Auditor receives Helper evidence stream or milestone packages.
- Auditor produces audit blueprints, tripwire requirements, gate checklists, and audit reports.
- Auditor gates only at audit points or risk triggers unless explicitly requested earlier.
- Auditor uses logical approval levels, not arbitrary scores.
- Auditor can require repair loops with Coder + Helper.

15. Audit Points

Audit points are deliberate places where Auditor involvement becomes necessary. They are not every step. They occur when the flow reaches a milestone, when a critical subsystem is implemented, when tripwire/guardrail production is needed, when deception risk appears, when recursive/self-modifying code is involved, or when claims/promotion are being considered.

Examples of audit points:

- after phase 4 in a 12-phase plan
- after phase 8 in a 12-phase plan
- after phase 12 final milestone
- before recursive self-improvement logic is accepted
- before a scientific claim is made
- after a tripwire failure
- before a critical subsystem is promoted
- when Helper or Coder behavior looks suspicious
- when evidence may be contaminated or non-replayable

User story

As the human operator, I want the PM flow plan to include explicit audit points, so Auditor is invoked at the right times instead of becoming a constant bottleneck.

Acceptance criteria

- PM plan includes audit points.
- Audit points include trigger condition and required evidence.
- Auditor receives enough context to set or apply gates.
- Flow cannot advance past an audit point without the required gate outcome or human override.

16. Approval Levels Instead of Auditor Scores

The user explicitly rejected the idea that an Auditor score such as 85/100 should determine advancement. The score was only an initial example. The better model is logical approval levels with explicit requirements.

Recommended approval levels:

Level 0 — Plan Ready

The plan is clear enough to start.

Requires:

- objective
- phases
- milestones
- definition of done
- allowed paths
- forbidden paths
- risks
- audit points
- gate criteria

Status:

`PLAN_READY`

Level 1 — Development Evidence Ready

Coder + Helper have produced runnable development evidence.

Requires:

- Coder patch bundle
- Helper execution report
- command logs
- exit codes
- tests/results
- no obvious forbidden path changes

Status:

`DEV_EVIDENCE_READY`

Level 2 — Integrity Gate Passed

Auditor has found the milestone evidence coherent enough to advance.

Requires:

- audit report
- evidence manifest
- tripwire/guardrail checks
- claim/evidence alignment
- no unresolved deception/gaming hard fail
- repair items resolved or explicitly deferred

Status:

`INTEGRITY_GATE_PASSED`

Level 3 — Claim or Promotion Candidate

The result may support a stronger claim or promotion decision but still requires human authority.

Requires:

- strict audit
- replayable evidence
- sealed/holdout boundary respected where relevant
- human review

- promotion/claim manifest
- no unresolved hard fails

Status:

`CLAIM_OR_PROMOTION_CANDIDATE`

User story

As the human operator, I want advancement to be governed by logical gate levels, not arbitrary scores, so each stage has evidence-appropriate approval criteria.

Acceptance criteria

- Auditor does not rely on a single numeric threshold.
- PM defines gate criteria in the flow plan.
- Auditor can refine audit requirements based on stage and risk.
- Human remains final promotion authority.

17. `/flow-status`

The `/flow-status` command should tell the human where the flow is without requiring manual repo inspection.

Example:

```
/flow-status name="flow00"
```

Expected summary:

- flow name
- flow ID
- type
- current phase
- current milestone
- latest actor
- next actor
- current gate level
- required next evidence
- blocking issues
- audit point status
- human action required or not

User story

As the human operator, I want `/flow-status` to summarize exactly where a flow stands, so I can decide whether to continue coding, invoke Auditor, advance the flow, or repair.

Acceptance criteria

- Flow can be loaded by `name` or `flow_id`.
- Status is concise but complete.
- Status identifies next actor and next required artifact.
- Status distinguishes informal notes from official artifacts.
- Status preserves conservative labels.

18. `/adv-flow`

The user wants `/adv-flow`, not `/advance-flow`, because the shorter command is easier to type. This command represents advancement of the flow after a gate or human decision.

Example:

```
/adv-flow name="flow00" decision="milestone-1-approved"
```

or:

```
/adv-flow name="flow00" to="phases-5-8"
```

The command should not blindly advance. The broker should check that required artifacts exist for the current gate. At high-risk gates, human approval should be required. If required artifacts are missing, the command fails closed.

User story

As the human operator, I want `/adv-flow` to move a flow to the next phase or milestone after required criteria are met, so advancement is explicit, auditable, and controlled by human judgment.

Acceptance criteria

- `/adv-flow` is short and easy to type.
- Advancement records a decision artifact or event.
- Required artifacts are checked before advancement.
- Missing evidence blocks advancement.
- Human can choose repair instead of advancement.
- Promotion-level advancement always requires human authority.

19. Proposed Flow Repository Structure

The flow system should use an explicit repo structure under ArqonZero.

Proposed layout:

```
governance/flows/
  flow_index.json

  FLOW-2026-0001/
    flow_manifest.json
    pm_plan.md
    milestone_map.md
    definition_of_done.md
    audit_points.md
    gate_criteria.md

    phases/
      phase_001.md
      phase_002.md

    milestones/
      milestone_001.md

    artifacts/
      coder_implementation_plan.md
      coder_patch_bundle.md
      coder_handoff_to_helper.md
      helper_execution_report.md
      evidence_manifest.json
      auditor_audit_blueprint.md
      auditor_gate_report.md
      human_decision.md
```

```
messages/
```

User story

As the system designer, I want each flow to have a predictable repo folder layout, so every role knows where to write and read official flow artifacts.

Acceptance criteria

- Flow index maps short names to canonical IDs.
- Each flow has a manifest.
- PM, Coder, Helper, Auditor, and Human artifacts have clear locations.
- Flow artifacts are separate from informal notes/messages.
- Forbidden paths remain blocked.

20. PM Sprint Story

The user's preferred daily pattern is to begin with PM GPT. The human discusses vision, mission, objectives, and the intended sprint for the day. PM turns this into a plan, milestone list, definitions, and flow-ready specifications.

The user then decides what flow type best fits the work. For safety-sensitive or governance-critical work, the default will usually be audit flow. For speed, milestone flow may be used. For high PM involvement, phase flow may be used.

The human issues:

```
/create-flow type=audit name="flow00"
```

The PM GPT writes the flow artifacts automatically and returns status. The next instruction is likely:

```
Open Coder GPT and run /load-flow name="flow00"
```

User story

As the human operator, I want PM GPT to turn a planning conversation into a flow-ready sprint with milestones and definitions, so I can hand the work to Coder without losing the strategic context.

Acceptance criteria

- PM creates comprehensive plan and milestones.
- PM defines audit points and gate criteria.
- PM writes artifacts automatically during flow creation.
- User chooses phase, milestone, or audit type.
- PM returns next command for Coder.

21. Coder + Helper Iteration Story

After the PM creates a flow, the human opens Coder GPT and loads the flow. Coder produces the implementation bundle and Helper packet. Then the human opens VS Code with Codex Helper.

Helper executes the packet and writes a report. That report goes back to Coder, not only to Auditor. Coder reads the Helper report and decides whether to repair, iterate, or continue to the next coding task. This is the fast development loop.

Auditor receives evidence or gets notified, but Auditor does not block every iteration unless the flow type or current gate requires it.

User story

As the human operator, I want Coder and Helper to iterate on code quickly while Auditor receives evidence for later gates, so development speed is preserved without losing auditability.

Acceptance criteria

- Coder writes Helper packet.
- Helper executes and reports.
- Helper report is available to Coder.
- Helper evidence is available to Auditor.
- Coder can repair without waiting for Auditor unless at audit point.
- Human decides when to continue or invoke an audit.

22. Audit Flow Story

In audit flow, PM creates the plan and audit criteria up front. Coder and Helper implement a milestone chunk. When the milestone reaches the audit point, Auditor audits the evidence, tripwires, guardrails, claims, and risk posture.

If Auditor finds issues, Coder and Helper iterate until the gate criteria are met. When the audit gate passes, Auditor messages the human or PM with a gate report. The human then decides whether to advance the flow using `/adv-flow` or send it back for additional work.

User story

As the human operator, I want audit flows to use Auditor where it matters most, so deception, adversarial weaknesses, evidence gaps, and claim inflation are caught at the right points without slowing every minor implementation step.

Acceptance criteria

- PM defines audit criteria.
- Auditor can create or require guardrails/tripwires.
- Helper evidence is available to Auditor.
- Auditor issues gate report, repair request, or hard-fail/quarantine recommendation.
- Human decides advancement.
- Audit pass does not equal promotion.

23. Design Implications for Implementation

The next implementation phase should not build `/create-run`. It should build Flow Core v0.3.

Recommended endpoints:

- `POST /v1/flows`
- `GET /v1/flows`
- `GET /v1/flows/{flow_ref}`
- `GET /v1/flows/{flow_ref}/status`

- `POST /v1/flows/{flow_ref}/artifacts`
- `POST /v1/flows/{flow_ref}/advance`

Where `flow_ref` can be either a canonical ID or a friendly name.

User-facing commands:

- `/create-flow type=audit name="flow00"`
- `/load-flow name="flow00"`
- `/flow-status name="flow00"`
- `/adv-flow name="flow00" decision="milestone-1-approved"`

Defaults:

- project defaults to ArqonZero
- role inferred from broker key
- type may default to audit

User story

As the system implementer, I want Flow Core v0.3 to implement natural flow objects instead of one-off runs, so the platform matches the user's real PM/Coder/Helper/Auditor workflow.

Acceptance criteria

- Flow creation writes PM artifacts.
- Flow loading returns role-specific view.
- Artifact writing is mostly automatic.
- Flow status is available.
- Flow advancement is explicit and gated.
- Friendly names resolve to canonical IDs.
- Role permissions are enforced.

24. Preserved Design Doctrine

The core doctrine that should be preserved in future sessions:

1. Do not force every step through Auditor.
2. Let Coder + Helper iterate quickly.
3. Give Auditor evidence and audit points.
4. Let PM plan whole sprints or milestone chunks up front.
5. Let the human choose phase, milestone, or audit flow.
6. Use friendly flow names.
7. Infer project and role by default.
8. Keep notes/messages informal.
9. Make artifacts official only through flow artifact paths.
10. Use gate levels, not arbitrary numeric audit scores.
11. Human decides advancement and promotion.
12. Required conservative status labels remain mandatory.

Status remains:

- REQUIRES_HUMAN_REVIEW
- development diagnostic only
- NOT SEALED-TEST CERTIFIED
- not promotable